# CI/CD PIPELINE WITH GITHUB ACTIONS & DOCKER

Internship Project Report | DevOps

| | | | |
|---|---|---|---|
| **Submitted by** | Karthigai Selvan M | **Project duration** | Internship project |
| **Source repository** | github.com/karthi2801/devops-cicd-project | **Container registry** | Docker Hub: karthi2801/devops-cicd-project |

## 1. Introduction

The project implements an automated Continuous Integration and Continuous Delivery (CI/CD) workflow for a containerized Python Flask web application. GitHub Actions is used to automate testing, Docker image creation and publishing. The published image is stored in Docker Hub and is then pulled and run locally using Docker. The implementation was completed on Windows using Git Bash and Docker Desktop.

## 2. Abstract

The objective is to reduce manual effort in validating and packaging an application. When code is pushed to the main branch, the workflow checks out the source, installs Python dependencies and runs automated tests. If the tests pass, a second job authenticates to Docker Hub, builds the container image and publishes it with a latest tag and a commit-specific tag. The published image was subsequently deployed as a Docker container and verified through the application and health endpoints.

## 3. Tools Used

| Tool / Technology | Purpose |
|---|---|
| Windows + Git Bash | Local development and command-line operations |
| Python 3.12, Flask, Gunicorn | Web application and production application server |
| Pytest | Automated endpoint tests |
| Docker, Docker Compose | Container image creation and local container management |
| Git and GitHub | Source control and code hosting |
| GitHub Actions | Automated CI/CD workflow execution |
| Docker Hub | Container image registry |

## 4. Steps Involved in Building the Project

- **Application development:** Created a Flask application with a home route (/) and a health-check route (/health).
- **Automated testing:** Added two Pytest test cases to verify the home response and health endpoint. Both tests passed.
- **Containerization:** Created a Dockerfile using Python 3.12 slim, installed the required packages, and configured Gunicorn to serve the application on port 5000. Docker Compose was used for local build and run operations.
- **CI workflow:** Configured GitHub Actions to run on pushes and pull requests targeting main, and through manual workflow dispatch. The test job checks out the code, sets up Python, installs requirements and runs Pytest.
- **Image publishing:** Added GitHub repository secrets for Docker Hub authentication. A dependent build-and-push job runs after successful tests on pushes to main, builds the image with Docker Buildx and publishes latest and commit-SHA tags.
- **Deployment and validation:** Pulled the published image from Docker Hub and ran it locally as cicd-production, mapping host port 5001 to container port 5000. The container was confirmed running, and both / and /health returned successful JSON responses.

## 5. Conclusion

The project successfully demonstrates an end-to-end CI/CD process for a Dockerized Flask application. The GitHub Actions workflow completed successfully, both automated tests passed, and the Docker image was published to Docker Hub. The published image was then deployed locally and verified through its application and health-check endpoints. This project provided practical experience with automated testing, containerization, image registry authentication, CI/CD workflow configuration and container deployment.

**Project links:** GitHub Repository | Docker Hub Image